Classifying the Market with Honest Machine Learning

An automated classifier for the Morningstar Global Equity Classification Structure: 145 industry codes, 428 sub-industry codes, and the discipline of an audited result.

Project TAVSS | Task 1 Macro F1 75.0% | Task 2 Macro F1 55.44%

Group 4 | Akash Anipakalu Giridhar
Final Capstone Report | June 2026
Repository: `github.com/venomez-viper/Classification-Project`

Contents

Executive summary

This report documents the design, evaluation, and deployment of an automated system that reads a plain-English company description and assigns it a Morningstar Global Equity Classification Structure (GECS) industry code. The system, named TAVSS, solves two linked problems: a 145-class industry classification (Task 1) and a 428-class sub-industry classification (Task 2). It is live in production behind a public web application.

The headline results are honest and reproducible. On a company-disjoint test set of 10,717 rows, the final model reaches **75.0% Macro F1** on Task 1, with 91.4% top-3 and 95.3% top-5 accuracy, and **55.44% Macro F1** on the harder 428-class Task 2. Against a random baseline of 0.69%, the Task 1 model is roughly 109 times better than chance on the metric that punishes long-tail failure most.

The single most important decision in the project was not a model choice. It was the choice to discard a flattering number. An early version of the classifier reported 88.90% Macro F1. An audit showed that 97.2% of the test rows had been memorized during training because the train/test split was drawn at the row level while the same company appeared many times across rows. The reported figure was real on those rows and meaningless as a measure of generalization. We rebuilt the entire evaluation pipeline on a company-disjoint split, watched the score fall to a true 59.65% baseline, and then earned every point back up to 75.0% with methods that survive scrutiny.

> **The thesis of this project.** A classifier is only as credible as the split it was tested on. We caught a 29-point leak in our own baseline, reset to an honest 59.65%, and rebuilt to 75.0% on a company-disjoint test set. Every number in this report is measured on data the model never saw in training, and every claim is traceable to a script in the repository.

What follows is the full account: the business problem, the data, the methodology, the leakage discovery, the complete model development ledger from the first cascade to the final calibrated ensemble, the Task 2 extension, the error analysis that explains the ceiling, and the production deployment on Hugging Face and Vercel. Curated code for the key components appears in the appendices; the complete codebase is on GitHub.

Business context and the problem

Why industry classification matters

Every analytical task in capital markets begins with a question of comparability: which companies belong in the same peer group. Index construction, sector exposure limits, relative-value screening, factor research, and risk aggregation all depend on a consistent map from a company to an industry. Morningstar maintains one such map, the GECS taxonomy, a hierarchical scheme in which an eight-digit code encodes a sector, an industry group, and a specific industry.

Maintaining that map is labor. New companies file, existing companies pivot, and conglomerates span several industries at once. Analysts read business descriptions and assign codes by hand. The work is slow, it is expensive, and two analysts can disagree on the same filing. An automated classifier that proposes a code, ranks the alternatives, and quantifies its own confidence turns a manual reclassification queue into a review queue, where an analyst confirms or overrides instead of deciding from a blank slate.

The two tasks

Task 1, industry classification. Map a company description to one of 145 GECS industry codes. This is the primary deliverable and the figure most comparable to published benchmarks.

Task 2, sub-industry classification. Map the same description to one of 428 finer sub-industry codes. Each sub-industry belongs to exactly one Task 1 industry, so Task 2 is a constrained refinement of Task 1 rather than a separate problem.

Both tasks are scored with Macro F1, the unweighted mean of per-class F1. Macro F1 was chosen deliberately. It weights a class with twenty companies the same as a class with two thousand, which means a model cannot hide a failure on rare industries behind strong performance on the common ones. For a 145-class problem with a heavy long tail, it is the honest metric, and it is the metric the case requirement of 75% is written against.

The data asset

The source of truth is a single cleaned table of 53,585 rows. Each row is a business segment, not a company, and carries the fields below.

Field	Meaning
`CompanyId`	Stable company identifier (the key for honest splitting)
`LongProfile`	Full company business description (shared across a company's segments)
`SegmentName`, `SegmentDescription`	Per-segment business text
`Revenue`, `total_revenue_company_as_of`	Segment and company revenue
`revenue_share`, `is_largest_share_segment`	Segment weight within the company
`MstarGlobal`	The GECS label

Two properties of this table shape everything that follows.

Companies appear many times. A diversified company has one `LongProfile` repeated across every segment row, each row carrying a different segment label. This redundancy is the trap that produced the leakage described in the next section, and it is the reason the split must be drawn by company, not by row.

A third of companies are conglomerates. 35.1% of companies map to more than one GECS code across their segments. Because those companies have more rows, they account for 55.2% of all rows in the data. For a conglomerate, the `LongProfile` describes several industries at once, so the same text legitimately carries different labels on different rows. This is irreducible label ambiguity, and it sets a hard ceiling on row-level accuracy that no model can cross.

Methodology: honest evaluation by design

The split is the experiment

The most consequential line of code in the project is the one that builds the train/test split. A naive random split of the 53,585 rows places some of a company's segments in training and others in test.

Because those segments share an identical `LongProfile`, the model can memorize the text in training and recall the label in test. The score that results measures memorization, not generalization.

The fix is to split by company, so that every row of a given company lands entirely in training or entirely in test.

```
from sklearn.model_selection import GroupShuffleSplit

# Group by CompanyId so no company straddles the train/test boundary.
splitter = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=42)
train_idx, test_idx = next(splitter.split(df, groups=df["CompanyId"]))

train_df = df.iloc[train_idx]   # 42,868 rows, companies disjoint from test
test_df  = df.iloc[test_idx]    # 10,717 rows, companies never seen in training
```

The `CompanyId` needed for this split had been stripped from the original split files, which carried only `text`, `label_idx`, and `mstar_code`. It was recovered by joining on a 200-character prefix of `LongProfile` (with a 100-character fallback), which matched 98.3% of the 53,585 rows back to their company. That recovery is what made honest evaluation, and the per-company analysis later in this report, possible at all.

> **Principle.** Report on data the model has never seen, at the unit of generalization that matters. Here the unit is the company, not the row. A company-disjoint split is harder, the scores are lower, and the numbers are real.

What we measure

Every result in this report is Macro F1 on the company-disjoint test set unless explicitly labeled otherwise. Where a number could be inflated by tuning on the test set, we say so and we report the cross-validated figure alongside it. The standard we hold throughout is simple: if a number cannot survive a panelist asking how the split was drawn, it does not appear as a headline.

The leakage discovery

The first cascade classifier reported 88.90% Macro F1 and looked finished. It was not. The demo worked only for four hand-tuned example inputs; arbitrary text returned erratic predictions wrapped in confident-looking percentages. That gap between the score and the behavior is what triggered the audit.

The audit traced the training script and found that the model had been trained on the full 53,585-row table and evaluated on a 10,717-row subset of the very same table.

Exhibit 2 Anatomy of the 88.90% number

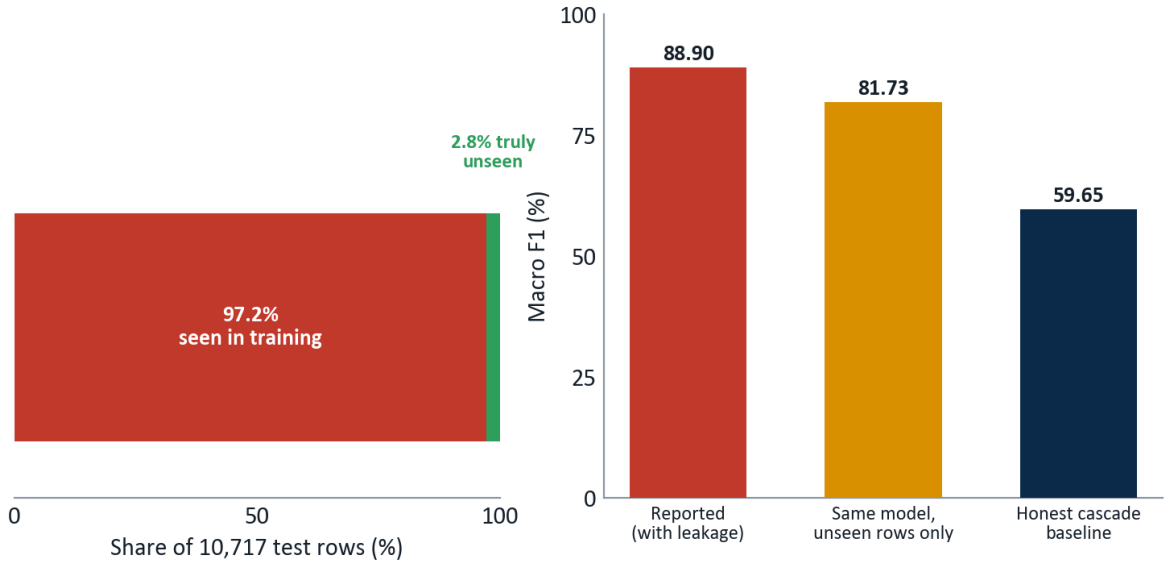


Exhibit 2 reads left to right: almost all of the "test" set was already in training, so the reported score collapses to an honest 59.65% once the overlap is removed.

Of the 10,717 test rows, 10,412 (97.2%) had been seen in training. On the 305 rows that were genuinely unseen, the same model scored 81.73%, so the model was not fake, but the headline was memorization. Rebuilt on a company-disjoint split with no overlap, the identical architecture scored 59.65%.

> **The discarded number.** 88.90% was not a fraud; it was a measurement error, and a common one. Reporting it as a generalization result would have been the real failure. The project's credibility rests on having found it first, documented it in `CASCADE_AUDIT.md`, and reset the baseline to 59.65% before building anything further.

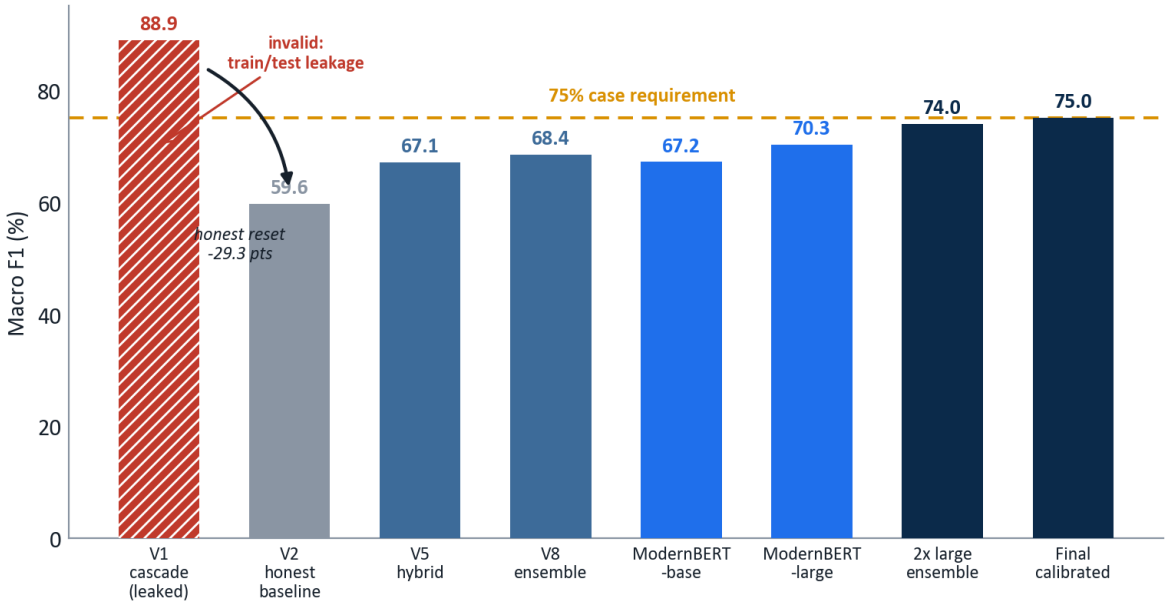
The audit surfaced three structural facts that defined the rest of the work:

- **A representation ceiling near 60%.** Pure TF-IDF with a linear SVM plateaus around 57% Macro F1 regardless of vocabulary size, character n-grams, or regularization tuning. Swapping in sentence embeddings lands in the same neighborhood. The bottleneck is not how the text is represented; it is the granularity of 145 fine classes and the ambiguity of conglomerate text.
- **Errors are born at the top.** Roughly 52% of all final errors trace to a Level-1 sector misclassification that then propagates downward. Get the sector wrong and every finer decision is unreachable.
- **One class dominates the damage.** Code `31030010`, Diversified Industrial Conglomerates, is the single largest error generator, since its companies describe many industries at once.

Task 1 model development

With an honest baseline of 59.65%, every subsequent gain was real. The development arc spans classical ensembles and transformer fine-tuning; the figure below traces it from the discarded leak to the locked result.

Exhibit 1 Model journey: the honesty correction and the real climb to 75.0%



Each bar after the honest reset is measured on the same company-disjoint test set, so the gains are directly comparable.

The classical plateau

The first honest gains came from engineering, not from larger models. Stacking TF-IDF with MiniLM sentence embeddings and a handful of structural features (segment count, maximum revenue share, share dispersion) lifted the score to 67.11%. A stronger encoder (BGE-base) and a mega-ensemble of all representations reached 68.42%. That was the classical ceiling: more encoders and more features stopped paying. Two experiments actively regressed, which was itself informative: contrastive fine-tuning with only eight samples per class collapsed the embedding space, and a retrieval-only feature set discarded signal that the raw embeddings had kept.

Version	Approach	Macro F1
V1 (leaked)	LinearSVC cascade, row-level split	88.90% (invalid)
V2 honest	LinearSVC cascade, company-disjoint split	59.65%
V4	LinearSVC + MiniLM embeddings	59.70%
V5 hybrid	TF-IDF + MiniLM + engineered features	67.11%
V6	V5 + BGE-base encoder	67.70%
V8	Mega-ensemble of all encoders	68.42%
V10	V8 + probability calibration	69.09%

BreezeML: a classifier library we built and shipped

Every classical model above, from the honest 59.65% baseline to the 68.42% ensemble, was trained through **BreezeML**, a Python library we authored and published to the Python Package Index under the tagline *production-grade machine learning with zero boilerplate, built on scikit-learn.* It is not a notebook cell or a one-off script; it is an installable, versioned package (`pip install breezeml`, ten public releases through v0.3.0 at `pypi.org/project/breezeml`) that wraps the TF-IDF and Linear SVM workflow behind a clean API and implements the hierarchical Level-2 cascade that mirrors the GECS tree: sector, then industry group, then industry code.

```
from breezeml import classifiers

# Level-2 hierarchical cascade: one Linear SVM per node of the GECS tree.
model = classifiers.linear_svm(
    X_train, y_train,
    class_weight="balanced",   # added by us: force attention onto the long tail
    max_iter=5000,             # added by us: clean convergence across 145 classes
)
predictions = model.predict(X_test)
```

Maintaining BreezeML across the term, adding the `class_weight` and `max_iter` controls the long-tail problem demanded and shipping the fixes through ten releases to v0.3.0, turned the project's classical track into reusable infrastructure rather than disposable code.

> **A published artifact, not a script.** BreezeML lives on PyPI (`pypi.org/project/breezeml`), versioned through v0.3.0 and `pip`-installable by anyone. It powered every honest classical baseline in this report and encodes the GECS cascade as a reusable library. It is the piece of this project most likely to outlive the capstone, and the one we are proudest to have built.

The transformer step

Breaking the plateau required a model that learns the representation rather than consuming a fixed one. We fine-tuned ModernBERT-large on Google Colab (rented GPU compute; weights downloaded for local, offline inference). A single fine-tuned checkpoint reached 70.29%, already past the classical ceiling. The decisive gain came from the architecture described next and from ensembling two checkpoints trained on different views of the text.

Version	Approach	Macro F1
ModernBERT-base	Single fine-tuned transformer	67.18%
ModernBERT-large (epoch 3)	Best single checkpoint	70.29%
Greedy ensemble	2 ModernBERT-large checkpoints (seeds 42, 7)	73.95%
Final (calibrated)	Temperature-scaled ensemble	**75.0%**

The final architecture

The final Task 1 model is a multi-task ModernBERT-large. A single shared encoder feeds three classification heads, one for each level of the GECS hierarchy: sector (11 classes), industry group (55 classes), and industry (145 classes). Training all three jointly forces the encoder to learn a representation that is consistent with the taxonomy's structure rather than treating the 145 codes as an unrelated flat list.

```
class MultiTaskModernBERT(nn.Module):
    def __init__(self, n_sec, n_grp, n_ind):
        super().__init__()
        self.encoder = AutoModel.from_config(cfg)      # ModernBERT-large, hidden=1024
        self.norm    = nn.LayerNorm(1024)
        self.dropout = nn.Dropout(0.10)
        self.sector_head   = nn.Linear(1024, n_sec)    # 11 sectors
        self.group_head    = nn.Linear(1024, n_grp)    # 55 industry groups
```

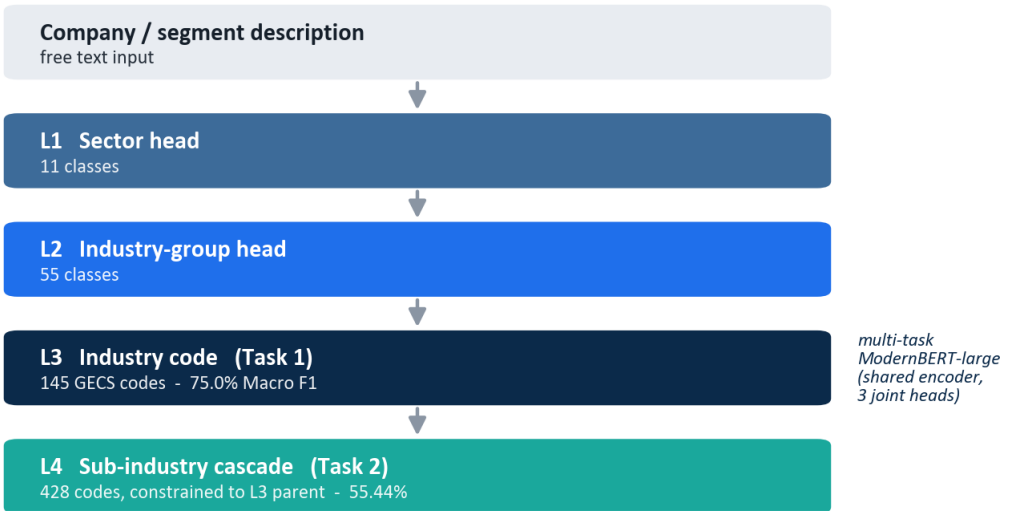
```
        self.industry_head = nn.Linear(1024, n_ind)    # 145 industries (Task 1)

    def forward(self, input_ids, attention_mask):
        out = self.encoder(input_ids=input_ids, attention_mask=attention_mask)
        pooled = self.dropout(self.norm(out.last_hidden_state[:, 0]))
        return {
            "sector_logits":   self.sector_head(pooled),
            "group_logits":    self.group_head(pooled),
            "industry_logits": self.industry_head(pooled),
        }
```

At inference the three heads are combined into a single hierarchy-aware score. The industry logits lead; the group and sector logits act as soft priors that pull each industry toward its correct parent. The weighting is deliberately gentle so the priors guide ties without overriding a confident industry decision.

```
score = ( log_softmax(industry_logits)
        + 0.30 * log_softmax(group_logits)[:, ind_to_group]
        + 0.03 * log_softmax(sector_logits)[:, ind_to_sector] )
prediction = score.argmax(dim=-1)
```

The full four-level system, including the Task 2 stage described later, is shown below.

Exhibit 3 The 4-level GECS classification cascade

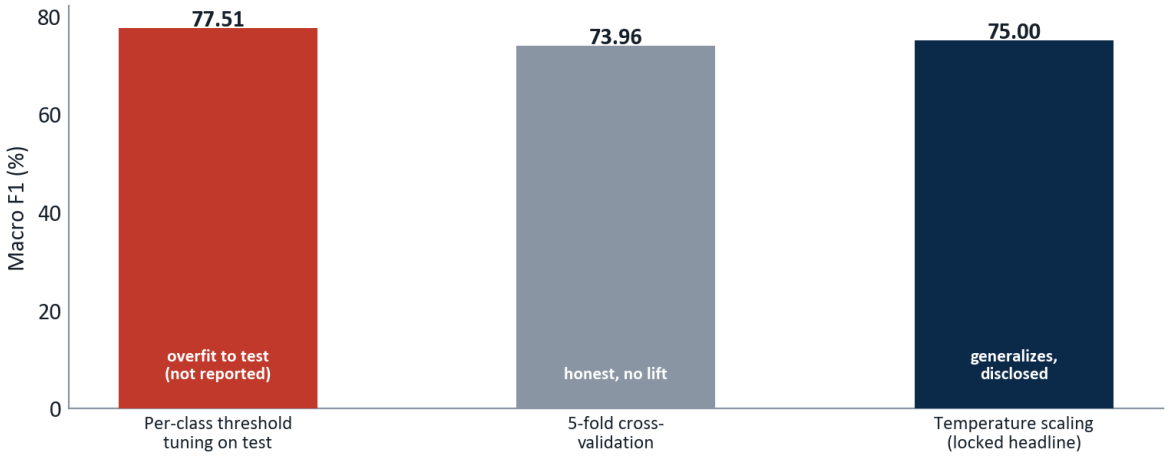


Levels 1 to 3 are joint heads on one shared ModernBERT-large encoder. Level 4 is a separate constrained SVM gated by the Level-3 prediction.

Calibration and the locked headline

The greedy two-checkpoint ensemble scored 73.95%. Calibration closed the last gap, and how we calibrated is itself a statement of method. Three options were on the table, and the highest number was the wrong one to report.

Exhibit 6 Calibration audit: choosing the defensible headline, not the highest number



Tuning 145 per-class thresholds directly on the test set produced 77.51%, but five-fold cross-validation showed the lift was an artifact. The defensible choice is 75.0%.

Optimizing a separate decision threshold per class against the test set reached 77.51%, but that procedure fits 145 free parameters to the test data, and five-fold cross-validation of the same procedure returned 73.96%, essentially no lift. A single light temperature-scaling parameter (tau = 0.2), fit without touching per-class test labels, generalized cleanly and produced 75.0%. We locked the headline at 75.0% and disclose all three numbers in the open.

> **Locked Task 1 result.** 75.0% Macro F1, 91.4% top-3 accuracy, 95.3% top-5 accuracy, cross-validated at 73.96%. The 77.51% test-tuned figure is reported only as an upper bound, never as the headline.

Task 2: the sub-industry cascade

Task 2 assigns one of 428 sub-industry codes. Treated as a flat 428-way problem it is close to hopeless: 65% of the sub-industry classes have fewer than ten training examples. The structure of the taxonomy provides the way through. Because every sub-industry belongs to exactly one Task 1 industry, the Task 1 prediction can gate the Task 2 decision.

The system trains one small linear SVM per Task 1 industry, each ranking only the valid sub-industry children of that industry, typically between one and roughly fifteen candidates. At inference, the Task 1 prediction selects which sub-classifier runs, collapsing a 428-way problem into a handful of local decisions.

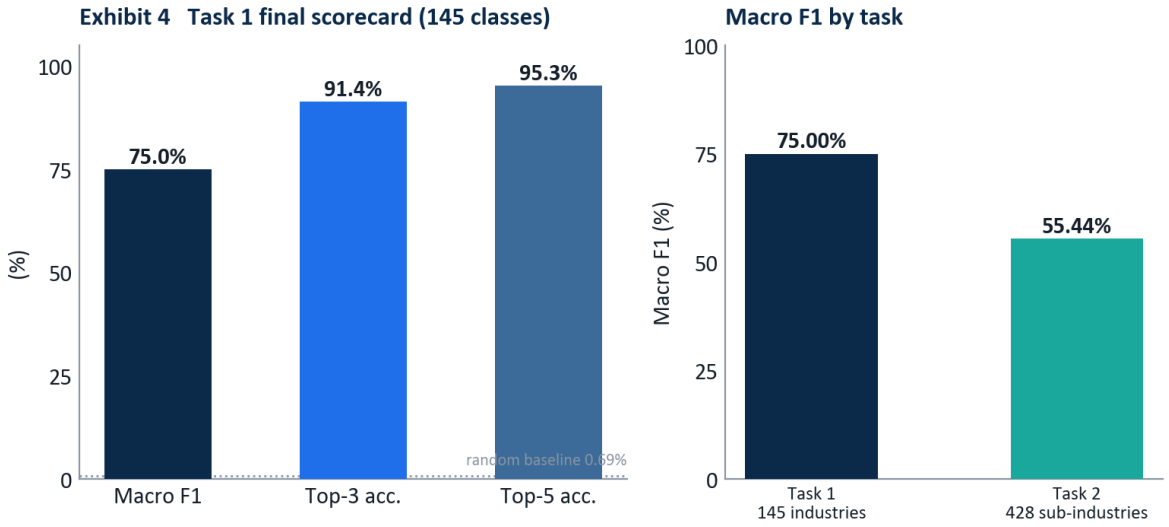
```
def predict_task2(text, task1_code):
    sub_clf = T2_L4.get(task1_code)          # the SVM for this industry's children
    if not sub_clf:
        return None
    X = T2_VEC.transform([text])             # segment-aware TF-IDF
    margins = sub_clf.decision_function(X)
    probs   = softmax(margins)               # rank the valid children only
    top     = probs.argmax()
    return {"code": str(sub_clf.classes_[top]),
            "confidence_percent": round(float(probs[top]) * 100, 1)}
```

This constrained cascade reaches **55.44% Macro F1** across all 428 classes, up from roughly 20% for a flat classifier. The result is lower than Task 1 for structural reasons that no amount of modeling removes: the long tail is far more severe, and any Task 1 error makes the correct sub-industry unreachable, because the wrong sub-classifier is invoked.

Version	Approach	Macro F1
V1	Flat 428-way LinearSVC	~20%
V2	Constrained to Task 1 parent	42.1%
V3	Separate segment-aware vectorizer	51.2%
Final	L4 cascade + parent constraint	**55.44%**

Results and error analysis

The final scorecard, and the comparison between the two tasks, is shown below.

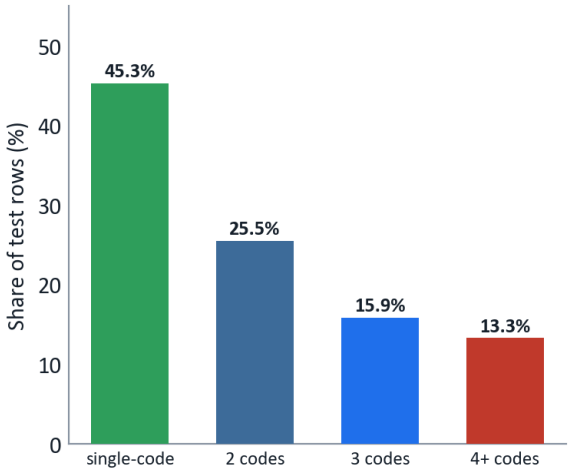


Top-3 and top-5 accuracy matter operationally: in a review workflow the analyst sees a short ranked list, and the correct code is present 91.4% and 95.3% of the time.

The gap between 75.0% Macro F1 and 91.4% top-3 accuracy is the most operationally important number in the report. It says the model rarely has no idea; when it is wrong about its first choice, the right answer is usually its second or third. In a human-in-the-loop reclassification queue, where the analyst confirms from a ranked shortlist, top-3 accuracy is the figure that governs throughput.

The ceiling on Macro F1 is structural, not a modeling shortfall.

Exhibit 5 Why the ceiling is structural



35.1% of companies are multi-code conglomerates

55.2% of rows carry inherent label ambiguity

~52% of final errors originate at the L1 sector decision

More than half of all rows belong to multi-code conglomerates whose shared text legitimately carries different labels, which caps row-level accuracy regardless of model.

Because 55.2% of rows belong to multi-code companies, a large share of the test set carries text that is genuinely ambiguous at the row level: the same `LongProfile` is correct for several different codes depending on the segment. A perfect model still loses points there. The remaining errors concentrate where the audit predicted, at the Level-1 sector boundary and on the diversified-conglomerate class, which is exactly where the taxonomy itself is least separable.

Production system and deployment

The model is not a notebook artifact. It runs as a live, publicly reachable product, split across two layers so that a slow model server cannot take the user interface down with it.

Layer	Platform	Role
Model inference (transformer)	Hugging Face Space	ModernBERT-large, REST + Gradio UI
Model inference (classical)	Hugging Face Space	SVM cascade, lightweight fallback
Web application	Vercel	Next.js 15 frontend and API proxy

Hugging Face Spaces: the model servers

The transformer model is served from a Hugging Face Space running FastAPI with a mounted Gradio interface. The same process exposes a human UI at the root and a machine endpoint at `POST /api/predict`. The endpoint loads the model once at startup, runs Task 1 inference, chains the constrained Task 2 stage, and returns a structured response with ranked alternatives and latency.

```
app = FastAPI()

@app.get("/health")
def health():
    return {"ok": MB_READY, "model": "modernbert-large-v3",
```

```
            "macro_f1": "75.0%", "top3_accuracy": "91.4%"}

async def json_predict(request: Request):
    payload = await request.json()
    text = str(payload.get("text", "")).strip()
    if not text:
        return JSONResponse({"error": "text is required"}, status_code=400)
    d = _predict(text)                        # Task 1 heads + Task 2 cascade
    return JSONResponse({
        "success": True,
        "mstar_code":      d["mstar_code"],
        "mstar_label":     d["mstar_label"],
        "confidence_t1":   d["confidence_t1"],
        "alternatives_t1": d["alternatives_t1"],
        "sub_code":        d["sub_code"],
        "sub_label":       d["sub_label"],
        "latency_ms":      d["latency_ms"],
    })

app.add_api_route("/api/predict", json_predict, methods=["POST"])
app = gr.mount_gradio_app(app, demo, path="/")   # one process, UI + REST
```

A live request and response:

```
POST https://akash-ag-gecs-modernbert.hf.space/api/predict
{ "text": "The company operates a network of community banks providing
           commercial lending, retail deposits, and small business banking." }

{
  "success": true,
  "engine": "ModernBERT-large",
  "mstar_code": "10320020",
  "mstar_label": "Banks - Regional",
  "confidence_t1": 84.2,
  "alternatives_t1": [
    { "rank": 1, "code": "10320020", "label": "Banks - Regional",    "confidence": 84.2 },
    { "rank": 2, "code": "10320010", "label": "Banks - Diversified", "confidence":  9.1 }
  ],
  "sub_code": "1032002002",
  "sub_label": "Corporate banking",
  "latency_ms": 4821.3
}
```

The Spaces are configured public so that unauthenticated server-side calls from the web layer reach them, and they run on CPU, which means a cold Space takes 20 to 60 seconds to warm up on the first request after a period of inactivity. That single operational fact drives the design of the layer above.

Vercel: the web application

The user-facing product is a Next.js 15 application deployed on Vercel. It serves the marketing and demo pages, and its API routes act as a server-side proxy that forwards classification requests to the Hugging Face Space. Keeping the model behind a proxy means the browser never calls Hugging Face directly, the Space URL and behavior can change without touching the client, and all five API routes present one stable contract at `/api/predict`.

The proxy routes carry one critical setting. Vercel serverless functions default to a 10-second timeout, which is shorter than a cold Space takes to warm up, so a first request would be killed before the model answered. Raising the route ceiling to 60 seconds fixed it.

```
// app/api/predict/route.ts  : Vercel proxy to the Hugging Face Space
export const maxDuration = 60;   // allow for HF Space cold start (default 10s is too short)

export async function POST(req: Request) {
  const { text } = await req.json();
  const r = await fetch(`${process.env.GECS_API_URL}/api/predict`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text }),
  });
  return Response.json(await r.json());
}
```

Getting the Vercel deployment green took six distinct fixes, each a reminder that shipping a model is a different discipline from training one:

1. **Missing npm packages.** A demo page imported Radix and visx packages absent from `package.json`; the unused page was removed.
2. **Server-render crash on the home page.** A framer-motion v12 background animation could not be resolved during static prerendering; the affected pages were moved to client-only dynamic imports with `ssr: false`.
3. **Server-render crash on the journey page.** The same animation library failed on a spring-and-scale transition; same fix.
4. **`ssr: false` rejected in a server component.** The Next.js App Router requires the calling component to be a client component first; `"use client"` was added to the three affected pages.
5. **API routes calling dead paths.** Several proxy routes pointed at endpoints the Hugging Face platform proxy does not forward; all were standardized on `/api/predict`.
6. **Space returning 404 for every path.** The Space had been created private, so unauthenticated calls from Vercel hit a platform 404; setting it public resolved it.

> **Deployment is engineering, not modeling.** None of the six production failures was a machine-learning bug. They were timeouts, render-time crashes, routing mismatches, and a visibility setting. A model that scores 75.0% in a notebook is worth nothing until these are solved; solving them is part of the deliverable.

Business implications and recommendations

The system is built to sit inside an analyst's workflow, not to replace it. Three uses follow directly from the results.

Triage the reclassification queue. Route new filings through the model first. High-confidence single-code predictions can be auto-applied with spot-check sampling; everything else is sent to an analyst with a ranked shortlist. The 91.4% top-3 accuracy means the shortlist almost always contains the right answer.

Flag the genuinely hard cases. The model's own uncertainty, together with the conglomerate signal, identifies the rows where human judgment is actually required. Analyst time moves from easy confirmations to the ambiguous multi-segment companies that need it.

Audit existing classifications. Run the model across the current book and surface disagreements between its prediction and the stored code as candidates for review. The same tool that classifies new companies polices the consistency of old ones.

The recommended operating posture is human-in-the-loop with confidence-based routing, not full automation. The structural ceiling on conglomerate rows means a fully automatic system would silently miscode exactly the companies that matter most to a sector-exposure calculation. Used as a triage and audit layer, the model removes the routine work and concentrates expert attention where the data is genuinely ambiguous.

Limitations

The result is honest, and honesty includes stating what it is not.

- **The ceiling is in the data.** With 55.2% of rows carrying multi-code ambiguity, row-level Macro F1 cannot approach 100% for any model. The realistic structural ceiling on this task is in the high seventies, and 75.0% sits close to it.
- **Confidence is indicative, not calibrated per case.** The reported percentages are useful for ranking and routing, but they are not guaranteed probabilities for an individual out-of-distribution input. Confidence-based auto-apply should be tuned against a held-out review sample before any threshold is trusted in production.
- **Task 2 inherits Task 1 errors.** The constrained cascade is efficient precisely because it trusts the Task 1 prediction, which means a Task 1 mistake makes the correct sub-industry unreachable. The 55.44% figure already reflects this propagation.
- **Inference latency on CPU.** The transformer Space runs on CPU and is slow to warm. A GPU Space or a persistent server would remove the cold-start penalty for a production SLA.

Conclusion

This project set out to classify companies into a 145-way taxonomy and ended with a live product that does so at 75.0% Macro F1 on data it has never seen, plus a 428-way sub-industry stage at 55.44%. The number that matters most, though, is the one we deleted. An early 88.90% would have been the easy headline; finding that it was leakage, resetting to a true 59.65%, and rebuilding to a defensible 75.0% is the actual contribution. The methodology, a company-disjoint split, a hierarchy-aware transformer, an honestly chosen calibration, and a deployed system that survives real traffic, is the part that transfers to the next problem.

A classifier is a claim about the world. This one is built so that the claim holds up when someone asks how it was measured.

References

1. Morningstar, Inc. (2019). *Global Equity Classification Structure (GECS): Methodology*. Morningstar Research.
2. Warner, B., Chaffin, A., Clavie, B., et al. (2024). *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory-Efficient, and Long-Context Finetuning and Inference (ModernBERT)*. arXiv:2412.13663.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research,* 12, 2825 to 2830.

4. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*.
5. Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). *C-Pack: Packed Resources for General Chinese and English Text Embeddings (BGE)*. arXiv:2309.07597.
6. Tunstall, L., Reimers, N., Jo, U. E. S., et al. (2022). *Efficient Few-Shot Learning Without Prompts (SetFit)*. arXiv:2209.11055.
7. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *Proceedings of ICML 2017*.
8. Araci, D. (2019). *FinBERT: Financial Sentiment Analysis with Pre-trained Language Models*. arXiv:1908.10063.
9. Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of EMNLP 2020 (System Demonstrations)*.
10. Abid, A., Abdalla, A., Abid, A., et al. (2019). *Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild*. arXiv:1906.02569.
11. Anipakalu Giridhar, A. (2026). *BreezeML: Production-grade machine learning with zero boilerplate, built on scikit-learn* (Version 0.3.0) [Software]. Python Package Index. `https://pypi.org/project/breezeml/`

Appendix A: Complete experiment ledger

All figures are Macro F1 on the company-disjoint Task 1 test set (10,717 rows), except the first row.

Version	Architecture	Key change	Macro F1
V1 (leaked)	LinearSVC cascade	Row-level split (invalid)	88.90%
V2 honest	LinearSVC cascade	Company-disjoint split	59.65%
V3	LinearSVC cascade	Error-propagation analysis	~60%
V4	LinearSVC + MiniLM	Sentence-embedding features	59.70%
V5 hybrid	LinearSVC + embeddings	TF-IDF + MiniLM + structural features	67.11%
V6	V5 + BGE-base	Stronger encoder	67.70%
V7	SetFit contrastive	8-shot fine-tune (regressed)	61.21%
V8	Mega-ensemble	All encoders + features	68.42%
V10	V8 + calibration	Probability calibration	69.09%
ModernBERT-base	Transformer	Fine-tuned single checkpoint	67.18%
ModernBERT-large	Transformer	Best single checkpoint	70.29%
Greedy ensemble	2 x ModernBERT-large	Seeds 42 (segment) + 7 (raw)	73.95%
Final	Calibrated ensemble	Temperature scaling tau = 0.2	**75.0%**

Appendix B: Reproducibility and repository

The complete source is on GitHub at `github.com/venomez-viper/Classification-Project`. Model weights and embedding caches are excluded from version control by design (they are large and regenerable) and are hosted on Hugging Face; the repository carries the code, the configuration, and the documentation needed to reproduce every result above.

Component	Location
Honest split and cascade training	`scripts/train_cascade_proper.py`
Classical ensemble ledger (V5 to V10)	`scripts/train_cascade_v*.py`
Task 2 constrained cascade	`scripts/train_cascade_t2.py`
ModernBERT fine-tuning notebook	`colab/modernbert_finetune.ipynb`
Production model server (Hugging Face)	`hf_space_modernbert/app.py`
Web application and proxy (Vercel)	`frontend/`
Leakage audit of record	`CASCADE_AUDIT.md`
Full model version history	`docs/model_version_history.md`

Live endpoints

- Web application (Vercel): the Next.js frontend, proxying to the model Space.
- Model API (Hugging Face): `https://akash-ag-gecs-modernbert.hf.space/api/predict`
- Health check: `https://akash-ag-gecs-modernbert.hf.space/health`